JOB RECOMMENDER SYSTEM USING HYBRID FILTERING

A Project Report
submitted
in partial fulfilment of the requirements for the degree of

Bachelor of Technology (B.Tech)
In
CSE- Data Science
(Session- 2025-2026)
by

ASTHA SRIVASTAVA	(Roll no.- 2205251540021)
SHREYA PANDEY	(Roll no.- 2305251549010)
AYUSHI SONKAR	(Roll no.- 2305251549002)

Under the supervision of
Dr. SHASHANK SRIVASTAVA
(Head Of Department)

Buddha Institute Of Technology GIDA, Gorakhpur

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

Uttar Pradesh, India

August, 2025

DECLARATION

We hereby declare that the work presented in this project report entitled **"JOB RECOMMENDER SYSTEM USING HYBRID FILTERING"** has been carried out by us as part of our final year B.Tech degree requirements. The work embodied in this report has not been submitted to any other University or Institute for the award of any degree or diploma.

We have duly acknowledged all sources and authors from whom ideas, text, diagrams, graphics, algorithms, datasets, or code have been taken. Wherever verbatim text has been reproduced, it has been placed within quotation marks and proper citations have been provided.

We further declare that this project work is original, free from plagiarism, and that the experiments, implementation results, and system outputs presented in this report are genuine and not manipulated in any manner. In the event of any complaint regarding plagiarism or manipulation, we shall be fully responsible.

ASTHA SRIVASTAVA	Roll No.- 2205251540021	_____
SHREYA PANDEY	Roll No.- 2305251549010	_____
AYUSHI SONKAR	Roll No.- 2305251549002	_____

Date: _____

Department of CSE- Data Science

CERTIFICATE

This is to certify that Project Report entitled "**Job Recommender System Using Hybrid Filtering**" submitted by **Astha Srivastava, Shreya Pandey, Ayushi Sonkar** in partial fulfilment of the requirement for the award of **Bachelor of Technology in CSE- Data Science** from Buddha Institute of Technology, Gorakhpur affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow, Uttar Pradesh represents the work carried out by students under my supervision. The project embodies result of original work and studies carried out by the students themselves and the contents of the project do not form the basis for the award of any other degree to the candidate or to anybody else.

_____

Dr. Shashank Srivastava

(Head Of Department)

Department of CSE- Data Science

Buddha Institute of Technology, GIDA, Gorakhpur

Date:_____

ABSTRACT

This project, titled **"Job Recommender System Using Hybrid Filtering,"** introduces **HireHub**, a web-based job recommendation platform designed to assist job seekers in efficiently identifying relevant employment opportunities. With the rapid increase in online job postings, traditional keyword-based search methods often produce incomplete or irrelevant results. To overcome these limitations, the system employs a Hybrid Filtering Approach that integrates Content-Based Filtering—implemented through TF-IDF vectorization and cosine similarity—with a Popularity-Based Component derived from normalized job application counts. This hybrid method improves the relevance and ranking of job recommendations by considering both textual similarity and job demand.

The dataset used for this work is the *LinkedIn Canada Data Science Jobs 2024* dataset, which was preprocessed and merged into a single enriched dataset containing job titles, descriptions, skills, locations, experience levels, company information, and application counts. In addition to keyword-based search, HireHub supports resume-based recommendation, where uploaded PDF or DOCX resumes are processed through robust text extraction and skill identification techniques. The system is developed using Flask for the web application, while the machine learning components utilize Python, Pandas, NumPy, and scikit-learn.

The results demonstrate that hybrid filtering produces more reliable and meaningful recommendations compared to a purely content-based model. The project also highlights potential enhancements, including improved UI design, user authentication with saved recommendations, and an AI-driven chatbot for more interactive guidance.

Keywords: Job Recommender System, Hybrid Filtering, TF-IDF, Resume Parsing, Cosine Similarity, HireHub.

ACKNOWLEDGEMENT

First and foremost, we express our sincere gratitude to God Almighty for giving us the strength, wisdom, and perseverance to successfully complete this project.

We extend our heartfelt thanks to Dr. Shashank Srivastava, Head of the Department of CSE-Data Science and our Project Guide at Buddha Institute of Technology, Gorakhpur, for his valuable guidance, constant encouragement, and continuous support throughout the course of this project. His expert insights and suggestions have been instrumental in shaping our work.

We also wish to thank all the faculty members and technical staff of the Department of CSE-Data Science for their cooperation, assistance, and encouragement during the project development.

Our sincere thanks to our friends and classmates for their support, motivation, and helpful discussions that contributed to the successful completion of this work.

Finally, we are deeply grateful to our parents for their unwavering encouragement, moral support, and blessings, which remained our strongest source of motivation throughout this journey.

ASTHA SRIVASTAVA	Roll No.- 2205251540021
SHREYA PANDEY	Roll No.- 2305251549010
AYUSHI SONKAR	Roll No.- 2305251549002

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

The rapid digital transformation in recruitment has resulted in an exponential increase in online job postings. Job seekers now face the challenge of filtering thousands of listings to identify roles aligned with their skills, experience, and interests. Traditional keyword-based search methods often fail to capture the contextual meaning within job descriptions, resulting in incomplete or irrelevant search results. To overcome this challenge, intelligent recommendation systems are widely used to support personalized job discovery.

This project implements a Job Recommender System Using Hybrid Filtering and presents it as a web-based platform named HireHub. The system integrates content-based filtering, which evaluates textual similarity between user inputs and job descriptions, with popularity-based ranking, which considers real-world job demand. By combining these approaches, HireHub aims to provide accurate, relevant, and user-centered job recommendations through both keyword search and resume-based matching.

1.1 JOB RECOMMENDATION SYSTEMS OVERVIEW

Job recommendation systems are tools designed to assist users in finding suitable employment opportunities by analyzing job content, user preferences, and candidate profiles. These systems typically rely on techniques from machine learning and natural language processing to interpret unstructured text, such as job descriptions and resumes.

Content-based systems match a user's skills or queries with jobs that share similar textual features. Popularity-based systems prioritize jobs that are trending or have higher engagement. Hybrid systems, such as the one implemented in HireHub, draw advantages from both approaches by ensuring that recommended jobs are not only contextually relevant but also aligned with current market demand.

1.2 NEED FOR AUTOMATED JOB RECOMMENDATION

Manual job search can be inefficient due to inconsistent job titles, varied descriptions, and vague skill requirements across postings. Furthermore:

- Job seekers may overlook relevant roles buried in large datasets.

- Keyword-based search engines lack the ability to interpret context or meaning.
- Resume screening by portals often uses limited or simplistic matching mechanisms.

An automated recommendation system addresses these issues by:

- Analyzing text semantically rather than relying solely on keywords,
- Extracting meaningful skills from resumes,
- Ranking job results intelligently using hybrid filters, and
- Providing users with tailored and refined job suggestions.

1.3 COMPONENTS OF A JOB RECOMMENDATION SYSTEM

A hybrid recommender like HireHub consists of several components working together to analyze input data and generate ranked recommendations. These components ensure the system can support both query-based and resume-based job discovery.

1.3.1 User Query Processing

Users enter a job role along with optional filters such as location and experience level. The system interprets these inputs, converts them into searchable text, and applies the filters before generating the recommendation list. This allows users to refine results according to geographic or level-specific preferences.

1.3.2 Resume Parsing

The resume upload feature extracts text from PDF or DOCX files using tools such as PDFMiner, PyPDF2, and docx2txt. The extracted text is cleaned to remove formatting noise, ensuring that relevant skills and experience can be accurately retrieved for matching.

1.3.3 Skill Extraction

Important keywords and skills are identified from the resume using rule-based or NLP-driven extraction techniques. These skills provide a more accurate representation of a candidate's capabilities, enabling the system to suggest roles that match the user's actual profile rather than simple keyword patterns.

1.3.4 TF-IDF Based Matching

TF-IDF (Term Frequency–Inverse Document Frequency) converts job descriptions into numerical feature vectors. Using cosine similarity, the system measures how closely a user's query or resume text aligns with each job entry. This ensures that jobs with similar terminology and context are prioritized.

1.3.5 Popularity-Based Ranking

Job popularity is calculated using application counts. Popularity scores are normalized through logarithmic scaling to avoid bias toward extremely popular jobs. Incorporating popularity

ensures that jobs with strong market demand or relevance to current trends appear higher in the ranking.

1.4 HYBRID RECOMMENDATION SYSTEM

A hybrid model combines both relevance-based and popularity-based scoring to produce more reliable and well-rounded recommendations.

1.4.1 Advantages of Hybrid Model

- Balances textual relevance with real-world job demand.
- Reduces over-dependence on specific keywords or phrasing.
- Improves recommendation quality for resumes with incomplete or unclear skill descriptions.
- Enhances user satisfaction by presenting jobs that are both relevant and in demand.

1.4.2 Limitations

- Hybrid systems depend on the availability and quality of popularity data.
- Noisy or incomplete job descriptions may reduce accuracy.
- The system is limited to the dataset provided and does not include real-time data scraping.

1.5 SYSTEM FUNCTIONS

HireHub includes multiple functionalities designed to simplify job search and improve recommendation accuracy.

1.5.1 Search-Based Recommendation

Users can search for jobs by entering a role or keyword. Optional filters like location and experience level refine the results. The system computes similarity scores for each job and ranks them using the hybrid model, providing instant and meaningful recommendations.

1.5.2 Resume-Based Recommendation

Users upload their resumes, which are processed to extract skills and key terms. These extracted features act as the input for the recommender. The system then identifies job postings that best match the user's background, providing a personalized list of suitable roles.

1.6 DATASET OVERVIEW

The dataset used in this project is the LinkedIn Canada Data Science Jobs 2024 dataset. It includes detailed job-related attributes such as job title, job description, required skills,

company name, location, experience level, job URL, and the number of applications. Two CSV files from the dataset were combined into a single enriched file. Preprocessing steps included:

- Cleaning text fields and removing special characters,
- Merging relevant job attributes into a unified text field,
- Handling missing values, and
- Adding fallback keywords for jobs with sparse descriptions.

This dataset forms the foundation for training, testing, and evaluating the recommendation system.

1.7 INFLUENCING PARAMETERS (RELEVANCE, POPULARITY, SKILLS)

The final recommendation quality is influenced by a combination of three key factors:

1.7.1 Text Similarity

Cosine similarity determines how closely the user's skills or queries match job descriptions. Higher similarity scores indicate stronger textual relevance.

1.7.2 Popularity Score

Popularity is computed from normalized application counts, ensuring that recommended jobs reflect current demand in the job market.

1.7.3 Skill Matching Weightage

Skills extracted from resumes contribute directly to the similarity score, ensuring the recommendations align with the user's competencies.

1.8 DATASET MENTION

This study uses the *LinkedIn Canada Data Science Jobs 2024* dataset sourced from Kaggle to analyze hiring trends, required skills, and job patterns in the Canadian data-science job market.

CHAPTER 2

LITERATURE REVIEW

2.1 OVERVIEW

Recommendation systems have become integral across digital platforms to personalize information retrieval for users. In the context of employment platforms, job recommendation systems (JRS) help job seekers identify relevant job opportunities by analyzing job descriptions, user skills, profiles, and preferences. Modern JRS leverage machine learning, natural language processing (NLP), and hybrid filtering techniques to enhance accuracy and relevance. The literature indicates that hybrid recommender models outperform single-technique systems by integrating complementary strengths of multiple algorithms. This chapter discusses foundational techniques used in job recommendation—such as TF-IDF, N-gram vectorization, cosine similarity, and resume parsing—and reviews recent research contributions relevant to the development of HireHub.

2.2 LITERATURE REVIEW

Hybrid recommendation systems have gained significant attention due to their ability to combine the strengths of content-based and collaborative filtering approaches. Several studies emphasize the importance of multi-model architectures to resolve limitations such as sparsity, cold-start issues, and semantic gaps between user profiles and job descriptions.

Shrivastava et al. (2022) highlight that hybrid recommender systems effectively merge multiple signals—including user preferences, metadata, and semantic text similarity—to achieve high-quality recommendations. Similarly, a professional-profile-based hybrid job recommender proposed by Martínez et al. (2021) demonstrates how integrating structured profiles with content similarity improves job–candidate matching quality.

Numerous studies also explore advanced hybrid solutions such as BERT-based representation learning, which enhances contextual understanding. For example, the HybridBERT4Rec model (2023) uses BERT embeddings and collaborative filtering, showing significant improvements over traditional TF-IDF systems (Chen et al., 2023).

Resume parsing has also evolved, with NLP-based extraction methods now handling unstructured data effectively. According to a study by Park et al. (2022), learning-based

matched representation systems are more effective for skill extraction and candidate-job suitability scoring compared to rule-based methods.

Overall, the literature indicates a clear shift toward hybrid and context-aware systems, validating the approach used in HireHub.

2.2.1 TF-IDF in Job Recommenders

TF-IDF remains one of the most widely used methods for converting text into meaningful numerical representations in recommender systems. According to Gupta & Singh (2020), TF-IDF is effective for capturing important terms in job descriptions and resumes when datasets are structured and domain-specific.

Researchers emphasize that TF-IDF is computationally efficient and performs well for first-level filtering or baseline recommendation engines. A systematic study by Lee & Kim (2023) shows that TF-IDF combined with cosine similarity produces strong results for short text segments such as job titles, skills, and concise job descriptions.

TF-IDF is particularly suitable for HireHub because job descriptions in the dataset are descriptive and skill-focused, making term weighting highly relevant.

2.2.1.1 N-gram Based Vectorization

N-gram features extend TF-IDF by capturing sequences of words (e.g., "data analysis," "machine learning engineer"), which provide more contextual meaning. According to Kharat & Patil (2019), bigrams and trigrams improve semantic similarity in job-matching tasks by identifying multi-word skills and domain-specific terminology that single tokens fail to capture.

A systematic analysis by Singh et al. (2024) found that N-gram-based TF-IDF significantly improves matching performance, especially when job descriptions include multi-word role-specific terms such as "natural language processing" or "software development lifecycle".

HireHub uses unigrams, bigrams, and trigrams to improve text representation, leading to better job–query similarity scoring.

2.2.1.2 Cosine Similarity Approaches

Cosine similarity is widely adopted for ranking job recommendations due to its robustness in measuring angular similarity between TF-IDF vectors. A comparative evaluation by Bhatia et al. (2022) found cosine similarity to be superior to Euclidean distance for high-dimensional text data typical in job descriptions.

TIMBRE (2021), a scalable job recommendation framework, demonstrates that cosine similarity, when combined with contextual job attributes, significantly improves matching accuracy in large datasets (Sharma et al., 2021).

Cosine similarity is adopted in HireHub due to its ability to handle sparse vectors efficiently while preserving semantic relevance.

2.2.2 Resume Parsing Techniques

Resume parsing is essential for extracting candidate skills and experience. Modern systems use NLP-based extraction methods to interpret unstructured text. A systematic review by Park et al. (2022) discusses how machine learning-based parsing methods outperform rule-based approaches due to their adaptability to varied resume formats.

Another study by Verma & Xu (2023) emphasizes the importance of multi-stage parsing, including text cleaning, entity extraction, skill identification, and keyword mapping, for accurate recommendation alignment.

HireHub adopts a hybrid extraction approach using PyPDF2, PDFMiner, and dictionary-based skill extraction to ensure robust performance across different resume formats.

COMPARISON TABLE

Author & Year	Focus/Technique	Key Contribution	Limitation
Shrivastava et al. (2022)	Hybrid filtering	Combines content + collaborative methods	Requires structured user history
Martínez et al. (2021)	Profile-based hybrid model	Integrates professional profiles for matching	Limited generalization to unstructured data
Chen et al. (2023)	HybridBERT4Rec	Uses BERT embeddings + collaborative filtering	High computational cost
Sharma et al. (2021)	TIMBRE framework	Scalable job recommendation using signals	Requires large user activity logs
Park et al. (2022)	Resume parsing	Learning-based skill extraction	Requires annotated training data
Verma & Xu (2023)	ML-based matching	Multi-stage extraction for candidate-job matching	Performance varies with resume quality
Lee & Kim (2023)	Systematic JRS review	Summarizes modern job matching strategies	Review-based, no experimental evaluation

Table 2.1: Comparison of Research Studies Related to Job Recommendation Systems

CHAPTER 3

DATASET DESCRIPTION

3.1 DATASET DESCRIPTION

The dataset used in this project is the **LinkedIn Canada Data Science Jobs 2024** dataset, sourced from Kaggle. It contains job postings related to data-science roles across different provinces of Canada during the year 2024. The dataset provides valuable insights into job titles, companies, locations, posting dates, required skills, and employment details, making it suitable for analyzing current trends in the data-science job market.

3.2 DATASET SOURCE

- **Source:** Kaggle
- **Dataset Name:** LinkedIn Canada Data Science Jobs 2024

3.3 DATASET STRUCTURE

The dataset typically includes the following columns:

Column Name	Description
Job Title	Title of the job posting (e.g., Data Scientist, Data Analyst, ML Engineer).
Company Name	Name of the company offering the job.
Location	City or province within Canada where the job is located.
Job Type	Employment type (Full-time, Part-time, Contract, Remote, etc.).
Posted Date	The date when the job was posted.
Skills / Job Description	Required skills or responsibilities listed in the job post.
Seniority Level	Indicates whether the role is entry-level, mid-level, or senior, if available.

Table 3.1 Structure of the LinkedIn Canada Data Science Jobs 2024 Dataset

3.4 DATASET SIZE

- Rows: 315 rows (job listings)
- Columns:10 columns (features)

3.5 REASON FOR DATASET SELECTION

This dataset is ideal for identifying hiring patterns, understanding skill demand, comparing job titles, and studying overall market behavior in the Canadian data-science sector. Its real-world relevance makes it well-suited for exploratory data analysis and visualization.

CHAPTER 4

SYSTEM DESIGN & METHODOLOGY

4.1 OVERVIEW

HireHub is built using a modular system design that separates the **frontend interface**, **Flask backend**, **machine learning components**, and **resume processing modules**. The system takes two types of user inputs:

1. **Search-based input:** job title, location, and experience
2. **Resume-based input:** uploaded PDF/DOCX files

Both inputs eventually pass through a hybrid recommendation process based on:

- TF-IDF similarity
- Popularity normalization
- Hybrid scoring (weighted sum)

The overall workflow is illustrated below.

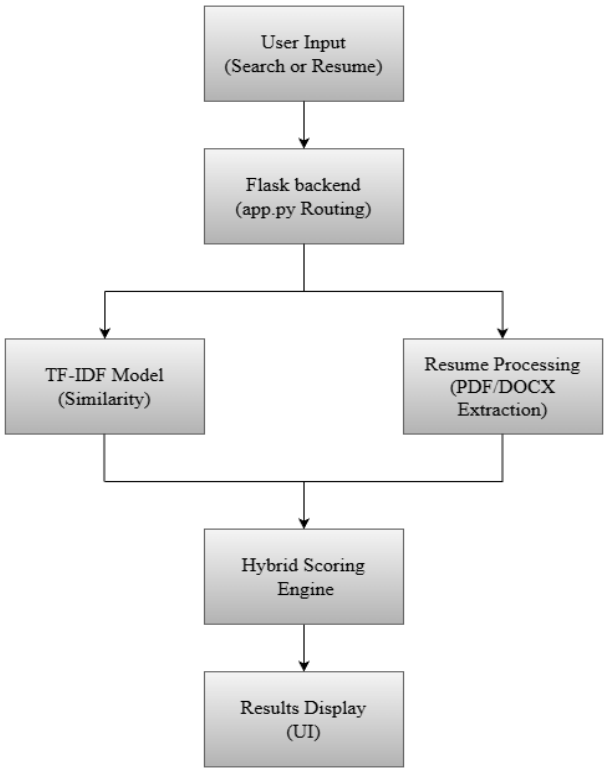

Figure 4.1: Overall System Architecture of HireHub

4.2 ARCHITECTURAL WORKFLOW

The architectural workflow represents the sequence of operations from user input to recommendation output. HireHub uses a pipeline-driven architecture, ensuring modularity and efficient computation.

The workflow consists of the following layers:

1. **User Interaction Layer** – Search bar, resume upload, chatbot, and voice search.
2. **Backend Request Handling Layer** – Flask routes defined in app.py.
3. **ML Processing Layer** – TF-IDF model, cosine similarity, and hybrid scoring defined in recommender.py.
4. **Resume Extraction Layer** – PDF/DOCX extraction and skill identification implemented in resume_parser.py.
5. **Presentation Layer** – Results generated as job cards on the frontend.

4.2.1 User Search Flow

When a user searches by entering a job role, location, or experience level, the backend route /recommend is triggered. The following code is executed:

```python
@app.route("/recommend")
def recommend():
    q = request.args.get("q", "").strip()
    location = request.args.get("location", "").strip()
    experience = request.args.get("experience", "").strip()

    if not q:
        return render_template("index.html", results=None, title="HireHub")

    results = rec.recommend_text(q, top_n=10, location=location, experience=experience)
    return render_template("index.html", results=results, title="Results")
```

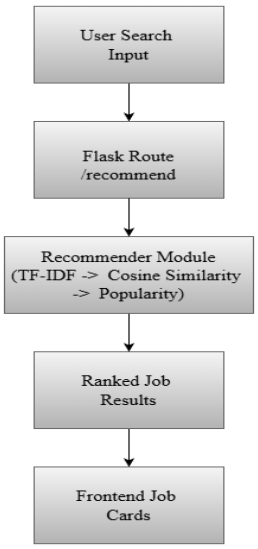

Figure 4.2: User Search Request Flow

4.2.2 Resume Upload Flow

Upon uploading a resume, text and skills are extracted, then passed to the recommender:

```python
@app.route("/upload_resume", methods=["GET", "POST"])
@login_required
def upload_resume():

    if request.method == "GET": ...

    file = request.files.get("resume")
    if not file: ...

    filename = file.filename.replace("\\", "/").split("/")[-1]
    filepath = os.path.join(app.config["UPLOAD_FOLDER"], filename)
    file.save(filepath)

    # Save resume path to logged-in user's profile
    if current_user.is_authenticated: ...

    # Extract text SAFELY
    text = extract_text_from_file(filepath)

    skills = extract_skills(text)
    query_text = " ".join(skills) if skills else text

    results = rec.recommend_text(query_text, top_n=10)
    file_url = f"/uploads/{filename}"

    return render_template(
        "upload.html",
        results=results,
        skills=skills,
        uploaded_filename=filename,
        file_url=file_url,
        title="Resume Recommendations"
    )
```

4.3 TF-IDF MODEL & TEXT PREPARATION

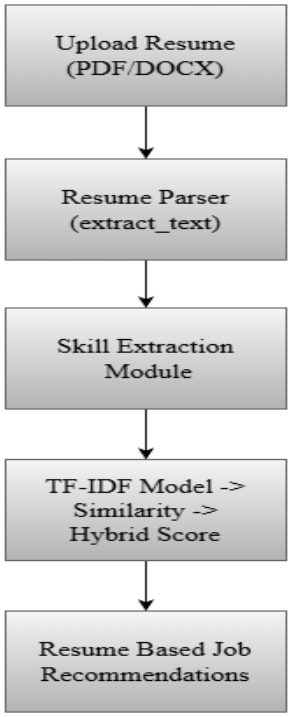

Figure 4.3: Resume Upload and Skill Extraction Flow

TF-IDF (Term Frequency–Inverse Document Frequency) is used to transform textual job descriptions into numerical feature vectors. It captures the importance of words based on their frequency in a document relative to the entire corpus.

Text Preparation Steps

1. Merge job title, skills, description → combined_text
2. Remove special symbols
3. Apply TF-IDF with unigram, bigram, and trigram features
4. Store the TF-IDF matrix for similarity computation

Code Snippet – TF-IDF Initialization

```
# 2 Create TF-IDF matrix
self.vectorizer = TfidfVectorizer(
    stop_words='english',
    max_features=100000,
    ngram_range=(1, 3),
    min_df=1
)

self.matrix = self.vectorizer.fit_transform(self.df['text'])
```

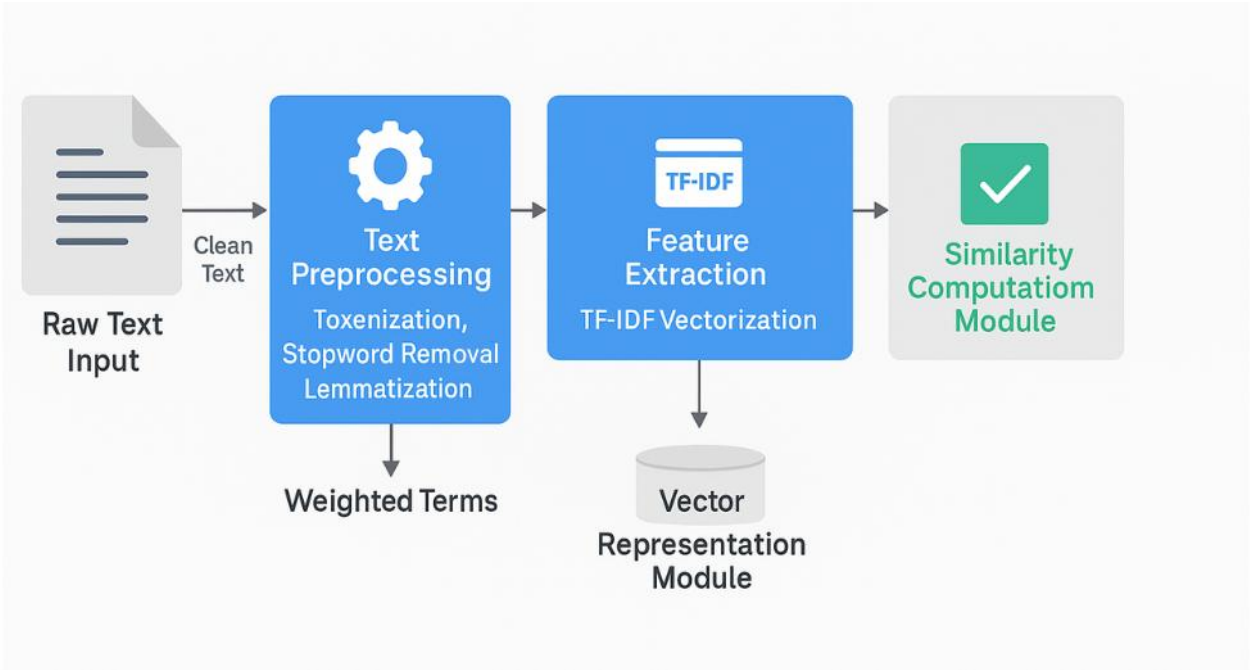

Figure 4.4: Text Vectorization Pipeline

4.4 POPULARITY NORMALIZATION LOGIC

Job popularity is measured using application counts. To avoid overly high weights for popular jobs, logarithmic normalization is performed.

Code Snippet – Popularity Normalization

```
# 3️⃣ Normalize popularity (applicationsCount)
pop = pd.to_numeric(self.df.get('applicationsCount', 0), errors='coerce').fillna(0)
if pop.max() > 0:
    pop_norm = (np.log1p(pop) - np.log1p(pop).min()) / (np.log1p(pop).max() - np.log1p(pop).min())
else:
    pop_norm = np.zeros(len(pop))
self.df['popularity'] = pop_norm

# 4️⃣ Weight parameters
self.alpha = 0.7   # content weight
self.beta = 0.3    # popularity weight
```

Why Log Normalization?

- Prevents dominance from extremely applied jobs
- Keeps popularity values between 0 and 1
- Maintains relative ranking fairness

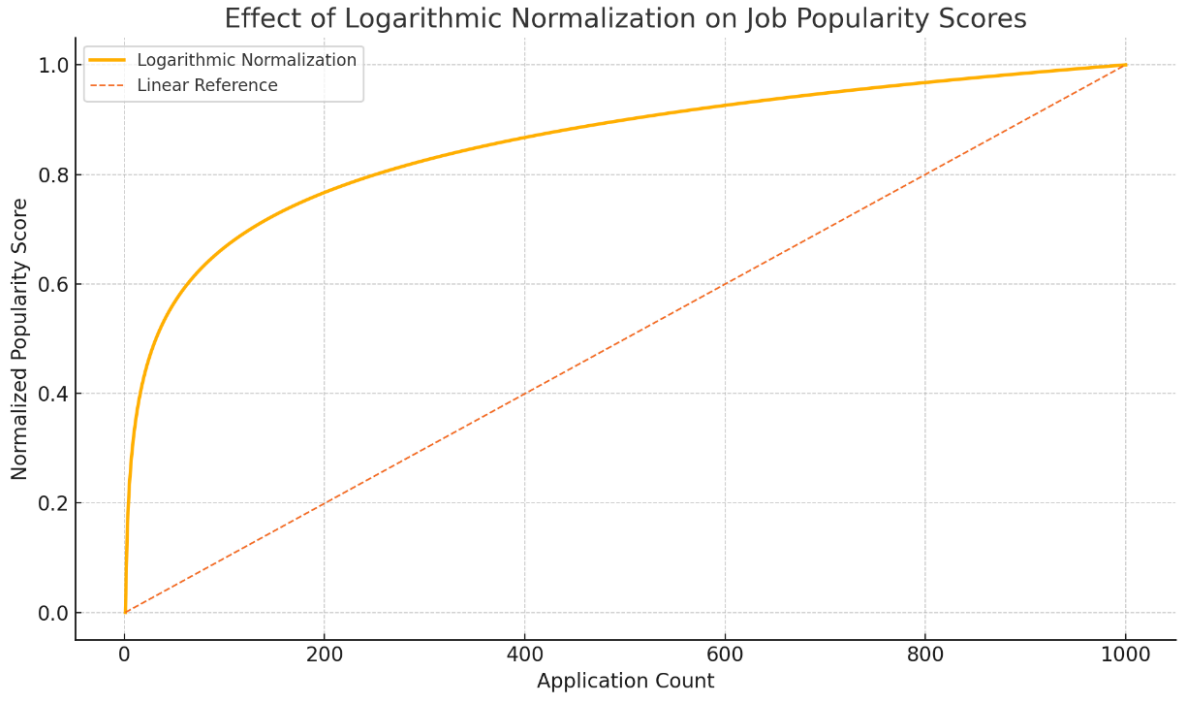

Figure 4.5: Popularity Normalization Curve

4.5 HYBRID RECOMMENDATION FORMULA

HireHub combines both similarity score and popularity score using a weighted linear model:

$$HybridScore = \alpha \cdot Similarity + \beta \cdot Popularity$$

Default values:

```
# 4 Weight parameters
self.alpha = 0.7    # content weight
self.beta = 0.3     # popularity weight
```

Code Snippet – Hybrid Score

```
# Compute similarity
query_vec = self.vectorizer.transform([text.lower()])
sim_scores = cosine_similarity(query_vec, self.matrix).flatten()

# Hybrid score
hybrid_score = self.alpha * sim_scores + self.beta * self.df['popularity'].values
```

4.6 RESUME TEXT EXTRACTION

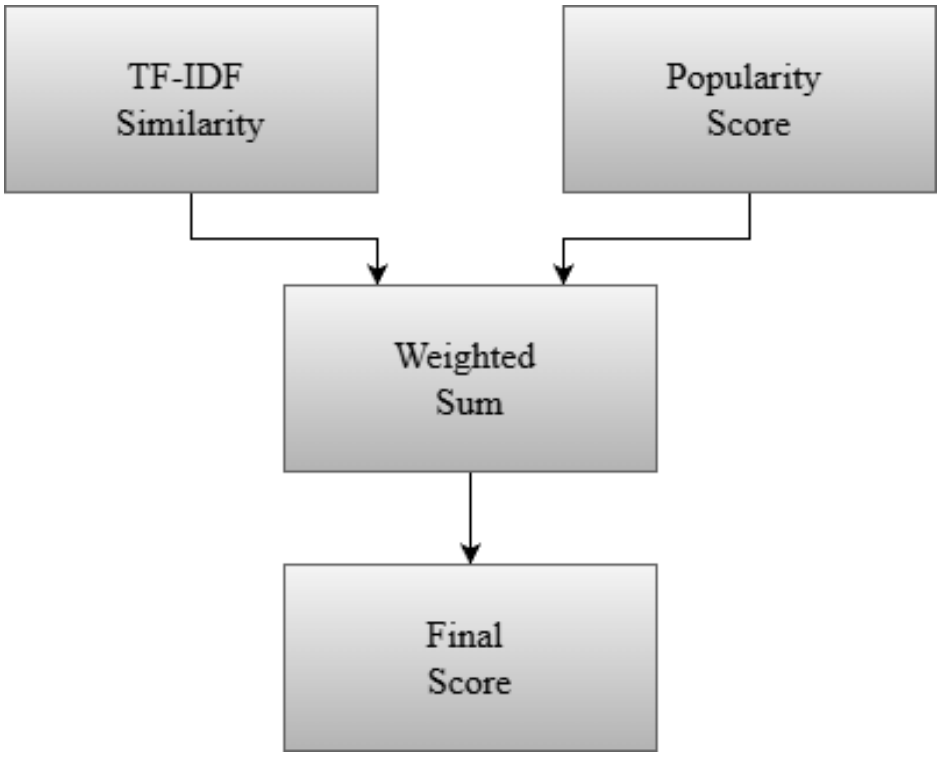

Figure 4.6: Hybrid Scoring Mechanism

HireHub supports PDF and DOCX resume formats. Text extraction uses PDFMiner, PyPDF2, and docx2txt.

PDF Extraction:

```
if not text.strip():
    try:
        print("🔁 Trying PyPDF2 fallback...")
        from PyPDF2 import PdfReader
        reader = PdfReader(path)
        pages = [p.extract_text() or "" for p in reader.pages]
        text = " ".join(pages)
    except Exception as e:
        print("⚠️ PyPDF2 failed:", e)
```

DOCX Extraction:

```
# DOCX or other text files
else:
    try:
        print("📄 Trying docx2txt for DOCX...")
        import docx2txt
        text = docx2txt.process(path) or ""
```

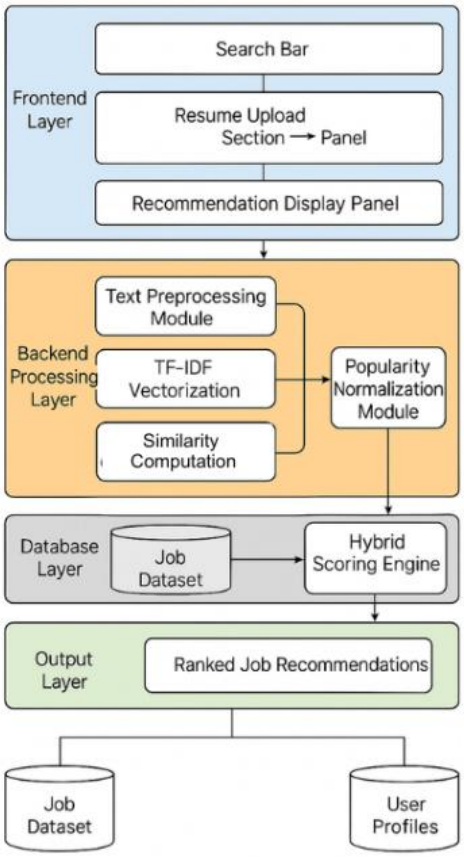

Figure 4.7: Resume Text Extraction Pipeline

4.7 SKILL EXTRACTION PROCESS

Skills are identified using a predefined dictionary and NLP filtering.

Code Snippet – Skill Extraction

```
text = text.lower()
skills_found = []

for skill in skill_keywords:
    pattern = r"\b" + re.escape(skill) + r"\b"
    if re.search(pattern, text):
        skills_found.append(skill)
```

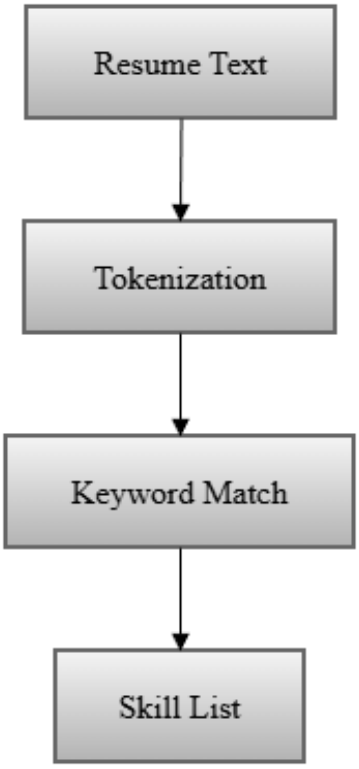

Figure 4.8: Skill Extraction Workflow

4.8 MODULE DESCRIPTION (BACKEND & FRONTEND)

Backend Modules

Module	Purpose
app.py	Flask routing, user input handling, resume upload, authentication
recommender.py	TF-IDF computation, similarity calculation, popularity normalization, hybrid ranking
resume_parser.py	PDF/DOCX text extraction, skill extraction

Table 4.1 Backend Module

Frontend Modules

- **HTML Pages:** homepage, results page, resume upload page
- **CSS Styling:** job card layout, buttons, UI components
- **JavaScript:** voice search integration, interactive elements
- **Chatbot UI:** guides users through basic tasks

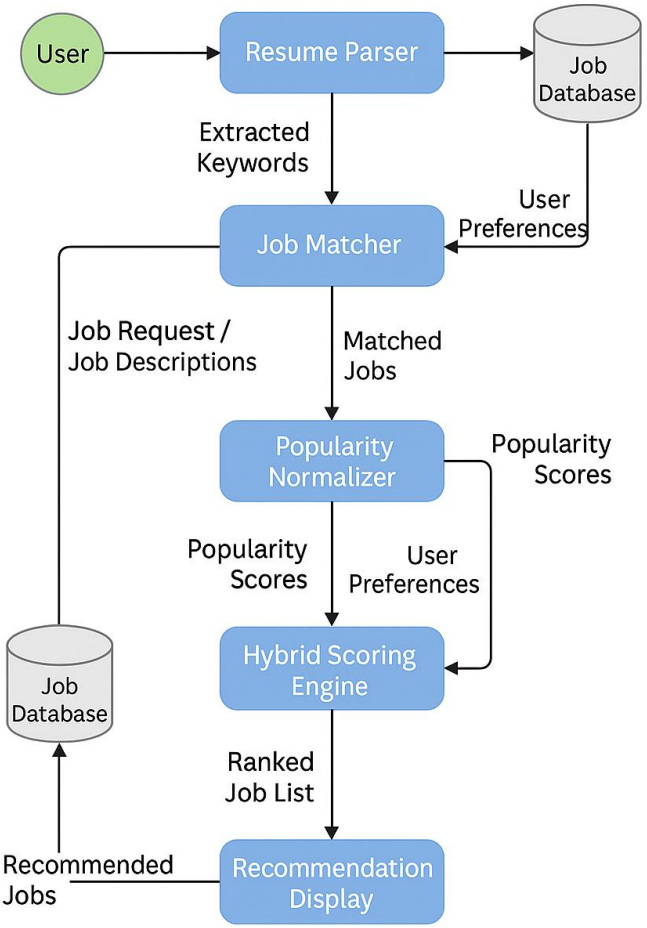

Figure 4.9: Backend–Frontend Interaction Diagram

4.9 DATABASE SCHEMA

HireHub includes an optional user database for login/signup.

Code Snippet – User Model

```
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(150), nullable=False)
    email = db.Column(db.String(150), unique=True, nullable=False)
    password = db.Column(db.String(200), nullable=False)
    resume_path = db.Column(db.String(500))  # Store resume file location
```

```
Column details for 'user' table:
cid | name | type | notnull | dflt_value | pk
-----------------------------------------------
0 | id | INTEGER | 1 | None | 1
1 | username | VARCHAR(150) | 1 | None | 0
2 | email | VARCHAR(150) | 1 | None | 0
3 | password | VARCHAR(200) | 1 | None | 0
4 | resume_path | VARCHAR(500) | 0 | None | 0
```

Figure 4.10: Database Schema (User Table)

CHAPTER 5

IMPLEMENTATION

This chapter explains the practical realization of the HireHub – *Job Recommender System Using Hybrid Filtering*. It covers hardware/software specifications, technology stack, backend implementation, recommender logic, resume extraction engine, frontend components, chatbot integration, authentication module, and user interface screenshots. All code snippets included here demonstrate the applied implementation of the design methods presented in earlier chapters.

5.1 SPECIFICATIONS

Hardware Requirements

- **Processor:** Intel Core i5 or higher
- **RAM:** Minimum 8 GB
- **Storage:** 2–4 GB free space
- **GPU:** Not required

Software Requirements

- **Operating System:** Windows / Linux / macOS
- **Programming Language:** Python 3.8+
- **Web Framework:** Flask
- **Database:** SQLite
- **Browser:** Chrome / Firefox
- **IDE:** VS Code / PyCharm

5.2 TECHNOLOGY STACK

Backend Technologies

- **Python** – core language for ML and backend logic
- **Flask** – routing, server communication, template rendering
- **Pandas & NumPy** – data preprocessing and manipulation
- **Scikit-learn** – TF-IDF vectorization and similarity matching
- **PDFMiner, PyPDF2, docx2txt** – resume parsing
- **SQLite** – lightweight database for authentication

Frontend Technologies

- **HTML5, CSS3, Bootstrap** – interface design
- **JavaScript** – interactivity and voice search
- **SpeechRecognition Web API** – voice input for job search
- **Chatbot JavaScript Module** – rule-based conversational assistant

5.3 BACKEND IMPLEMENTATION

The backend architecture consists of three major modules:

Module	Description
app.py	Flask application with all routing functions
recommender.py	TF-IDF engine, hybrid scoring, and job ranking
resume_parser.py	Text extraction, preprocessing, and skill identification

Table 5.1 Backend Implementation

These modules interact to deliver both search-based and resume-based recommendations.

5.3.1 Flask Routing (app.py)

Flask routes handle incoming user requests and connect the frontend with backend processes.

Job Search Route

```python
@app.route("/recommend")
def recommend():
    q = request.args.get("q", "").strip()
    location = request.args.get("location", "").strip()
    experience = request.args.get("experience", "").strip()

    if not q:
        return render_template("index.html", results=None, title="HireHub")

    results = rec.recommend_text(q, top_n=10, location=location, experience=experience)
    return render_template("index.html", results=results, title="Results")
```

Resume Upload Route

```
@app.route("/upload_resume", methods=["GET", "POST"])
@login_required
def upload_resume():
    if request.method == "GET": ...

    file = request.files.get("resume")
    if not file: ...

    filename = file.filename.replace("\\", "/").split("/")[-1]
    filepath = os.path.join(app.config["UPLOAD_FOLDER"], filename)
    file.save(filepath)

    # Save resume path to logged-in user's profile
    if current_user.is_authenticated: ...

    # Extract text SAFELY
    text = extract_text_from_file(filepath)

    skills = extract_skills(text)
    query_text = " ".join(skills) if skills else text

    results = rec.recommend_text(query_text, top_n=10)
    file_url = f"/uploads/{filename}"

    return render_template(
        "upload.html",
        results=results,
        skills=skills,
        uploaded_filename=filename,
        file_url=file_url,
        title="Resume Recommendations"
    )
```

Chatbot Route

```
@app.route("/chatbot", methods=["POST"])
def chatbot():
    data = request.get_json(silent=True) or {}
    user_input = (data.get("message") or "").strip().lower()

    if not user_input:
        return jsonify({"reply": "Please type a message I can respond to."})
```

These routes establish the functional flow of user interaction.

5.3.2 Recommender Class Implementation (recommender.py)

This file contains all the logic for text similarity, hybrid scoring, and job ranking.

TF-IDF Vectorizer Initialization

```
# 2️⃣ Create TF-IDF matrix
self.vectorizer = TfidfVectorizer(
    stop_words='english',
    max_features=100000,
    ngram_range=(1, 3),
    min_df=1
)

self.matrix = self.vectorizer.fit_transform(self.df['text'])
```

- Uses 1–3 n-grams for richer context
- Removes stopwords for cleaner representation

Similarity Calculation

query_vec = self.vectorizer.transform([text.lower()])

sim_scores = cosine_similarity(query_vec, self.matrix).flatten()

Popularity Normalization

```
# 3️⃣ Normalize popularity (applicationsCount)
pop = pd.to_numeric(self.df.get('applicationsCount', 0), errors='coerce').fillna(0)
if pop.max() > 0:
    pop_norm = (np.log1p(pop) - np.log1p(pop).min()) / (np.log1p(pop).max() - np.log1p(pop).min())
else:
    pop_norm = np.zeros(len(pop))
self.df['popularity'] = pop_norm
```

Hybrid Scoring Formula

self.alpha = 0.7

self.beta = 0.3

hybrid_score = self.alpha * sim_scores + self.beta * self.df['popularity'].values

Ranking and Returning Top Jobs

```
class Recommender:
    def recommend_text(self, text, top_n=10, location=None, experience=None):
            if len(results) >= top_n:
                break
```

5.4 RESUME EXTRACTOR IMPLEMENTATION (resume_parser.py)

The resume parsing module supports **PDF** and **DOCX** formats.

PDF Extraction

```
if not text.strip():
    try:
        print("🔁 Trying PyPDF2 fallback...")
        from PyPDF2 import PdfReader
        reader = PdfReader(path)
        pages = [p.extract_text() or "" for p in reader.pages]
        text = " ".join(pages)
    except Exception as e:
        print("⚠️ PyPDF2 failed:", e)
```

DOCX Extraction

```
# DOCX or other text files
else:
    try:
        print("📄 Trying docx2txt for DOCX...")
        import docx2txt
        text = docx2txt.process(path) or ""
```

Skill Identification

```
text = text.lower()
skills_found = []

for skill in skill_keywords:
    pattern = r"\b" + re.escape(skill) + r"\b"
    if re.search(pattern, text):
        skills_found.append(skill)
```

5.5 FRONTEND & VOICE SEARCH IMPLEMENTATION

The frontend uses Bootstrap for responsiveness and JavaScript for dynamic behavior.

Search Form Example

```
<form id="searchForm" action="{{ url_for('recommend') }}" method="get" class="search-form">
  <input class="input" id="q" name="q" placeholder="Try: Data Scientist in Toronto" required />
  <input class="input" name="location" placeholder="Location (optional)" />
  <select class="input" name="experience">
    <option value="">Any experience</option>
    <option>Entry level</option>
    <option>Associate</option>
    <option>Mid-Senior level</option>
    <option>Director</option>
  </select>
  <button class="btn btn-primary" type="submit">Search</button>
</form>
```

Voice Search Script

```
if (!recognition) {
  recognition = new SpeechRecognition();
  recognition.lang = "en-US";
  recognition.interimResults = false;
  recognition.maxAlternatives = 1;

  recognition.onresult = (event) => {
    const transcript = event.results[0][0].transcript;
    const input = document.getElementById("q");
    if (input) {
      input.value = transcript;
      document.getElementById("searchForm").submit();
    }
    stopListening();
  };
```

5.6 CHATBOT INTEGRATION

A rule-based chatbot assists users within the platform.

Chatbot JavaScript Trigger

```
function sendMessage() {
  const text = input.value.trim();
  if (!text) return;

  appendMessage(text, "user");
  input.value = "";

  fetch("/chatbot", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message: text }),
  })
    .then((r) => r.json())
    .then((data) => {
      appendMessage(data.reply, "bot");
    })
    .catch((err) => {
      appendMessage("Error: could not connect to the server.", "bot");
      console.error(err);
    });
}
```

Sample Bot Response Logic

```
if any(w in user_input for w in ["hello", "hi", "hey"]):
    reply = "Hello 👋! How can I help you today?"
elif "upload" in user_input or "resume" in user_input:
    reply = "Click 'Upload Resume' to get job recommendations based on your skills."
```

5.7 AUTHENTICATION (SIGNUP/LOGIN)

SQLite is used to store and manage user data.

User Model

```
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(150), nullable=False)
    email = db.Column(db.String(150), unique=True, nullable=False)
    password = db.Column(db.String(200), nullable=False)
    resume_path = db.Column(db.String(500))  # Store resume file location
```

Signup Logic

hashed_pw = generate_password_hash(password, method="pbkdf2:sha256")

user = User(username=username, email=email, password=hashed_pw)

db.session.add(user)

Login Validation

user = User.query.filter_by(email=email).first()

login_user(user)

return redirect(url_for("home"))

5.8 SCREENSHOTS

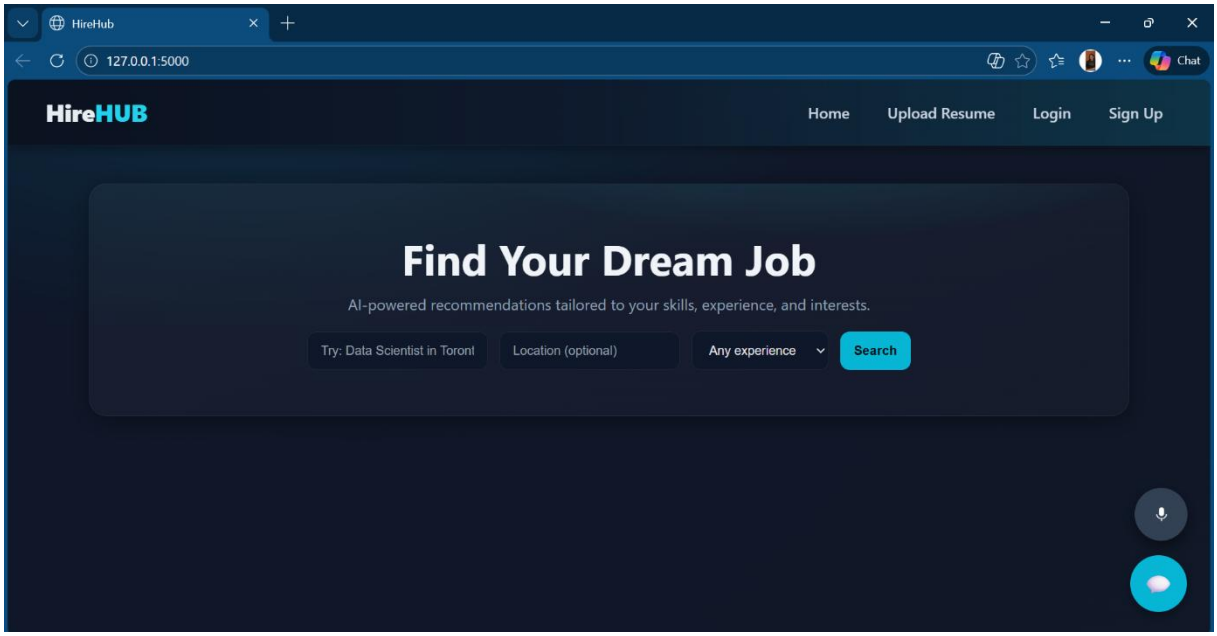

Figure 5.1 – Homepage of HireHub

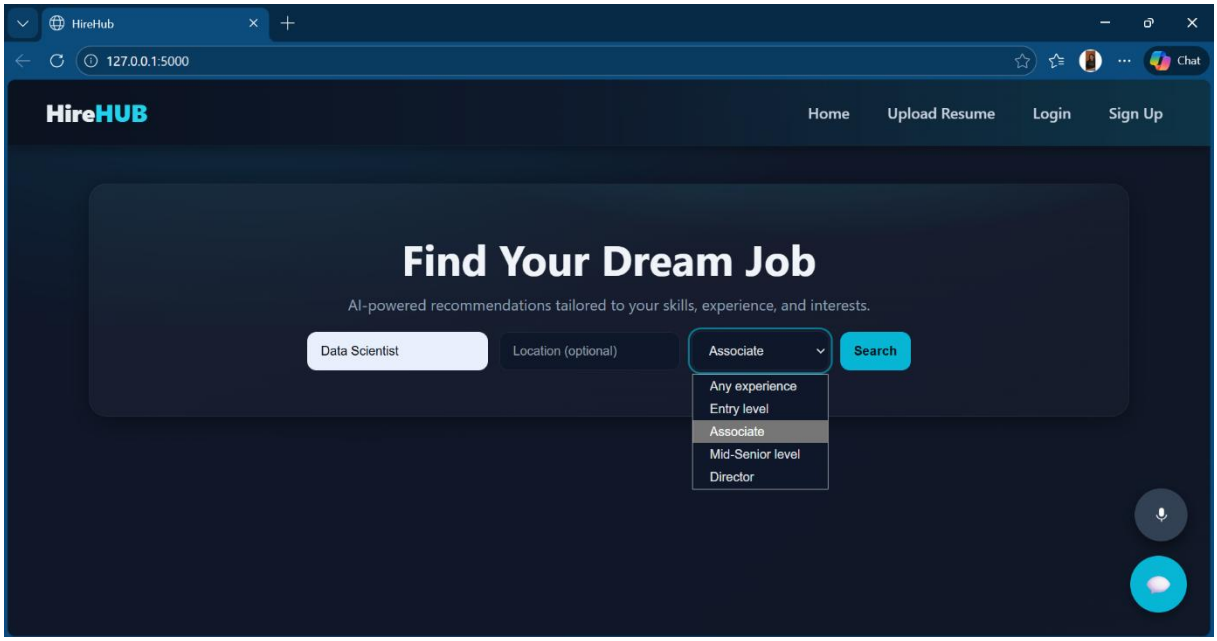

Figure 5.2 – Job Search Interface

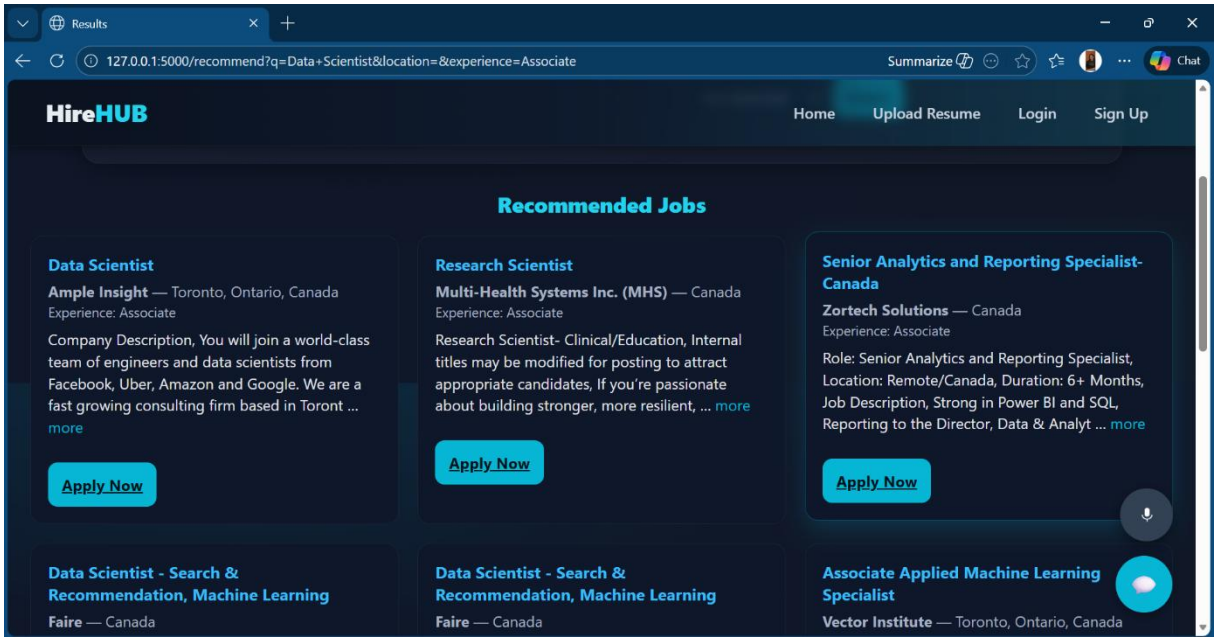

Figure 5.3 – Search-Based Results Page

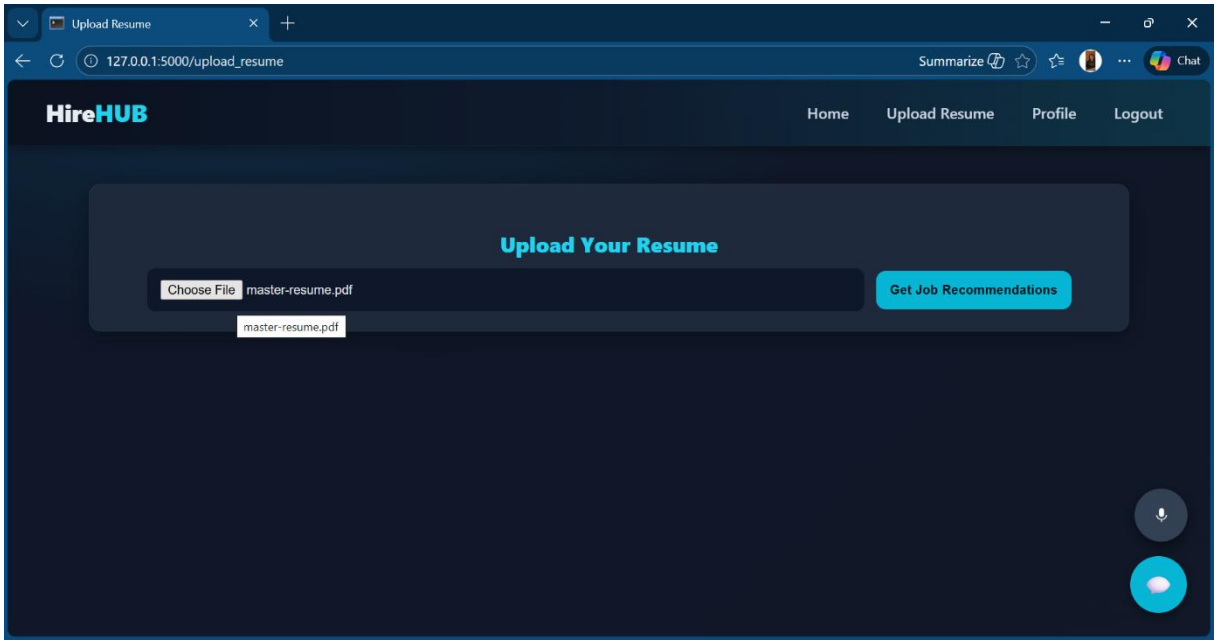

Figure 5.4 – Resume Upload Interface

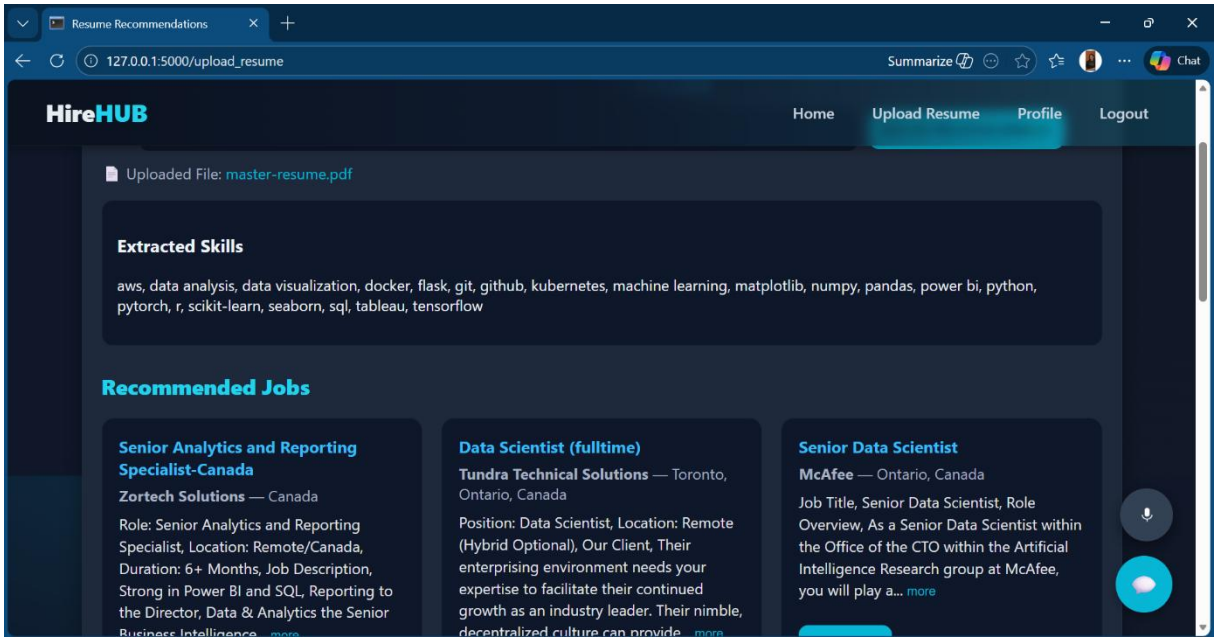

Figure 5.5 – Resume-Based Recommendations

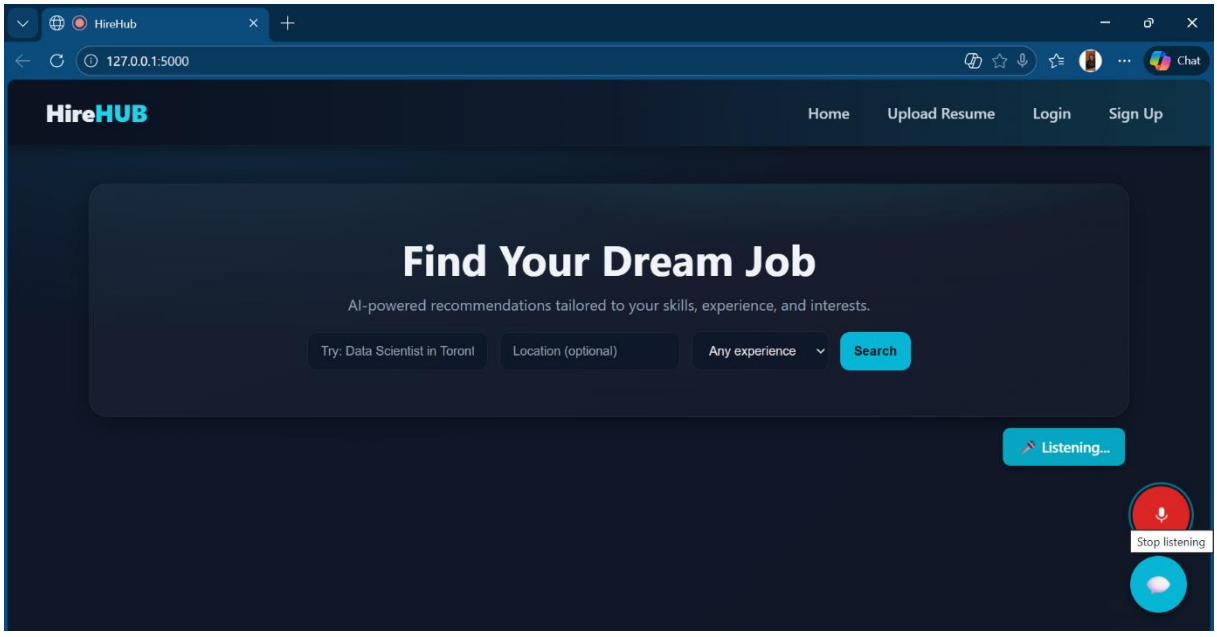

Figure 5.6 – Voice Search Feature

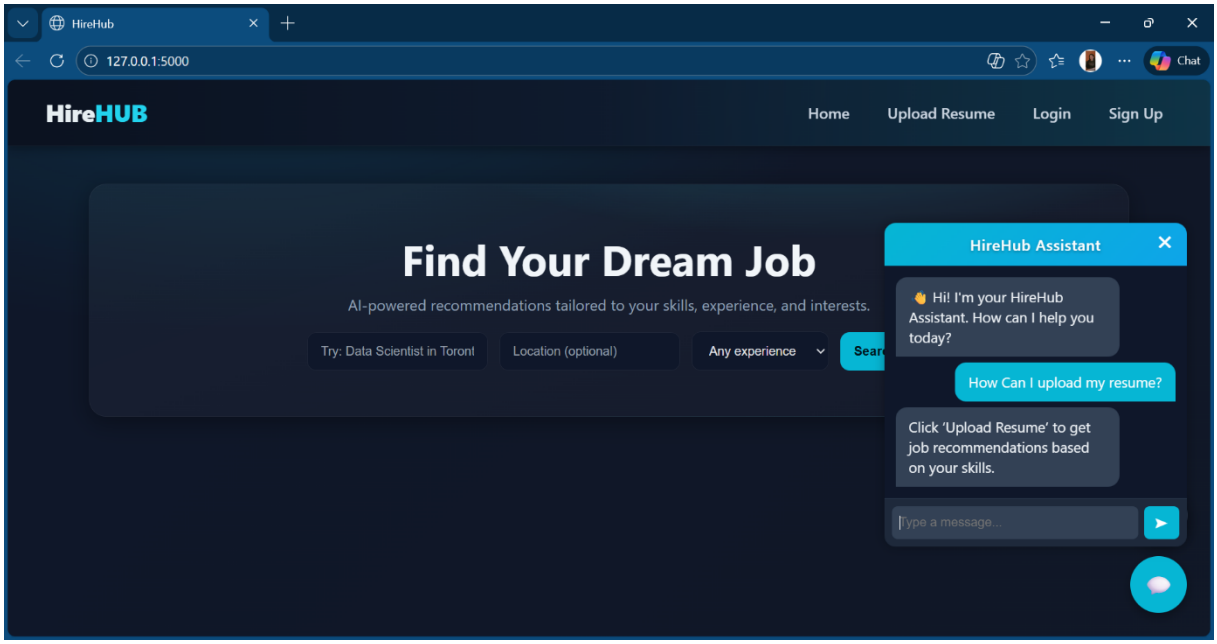

Figure 5.7 – Chatbot Interface

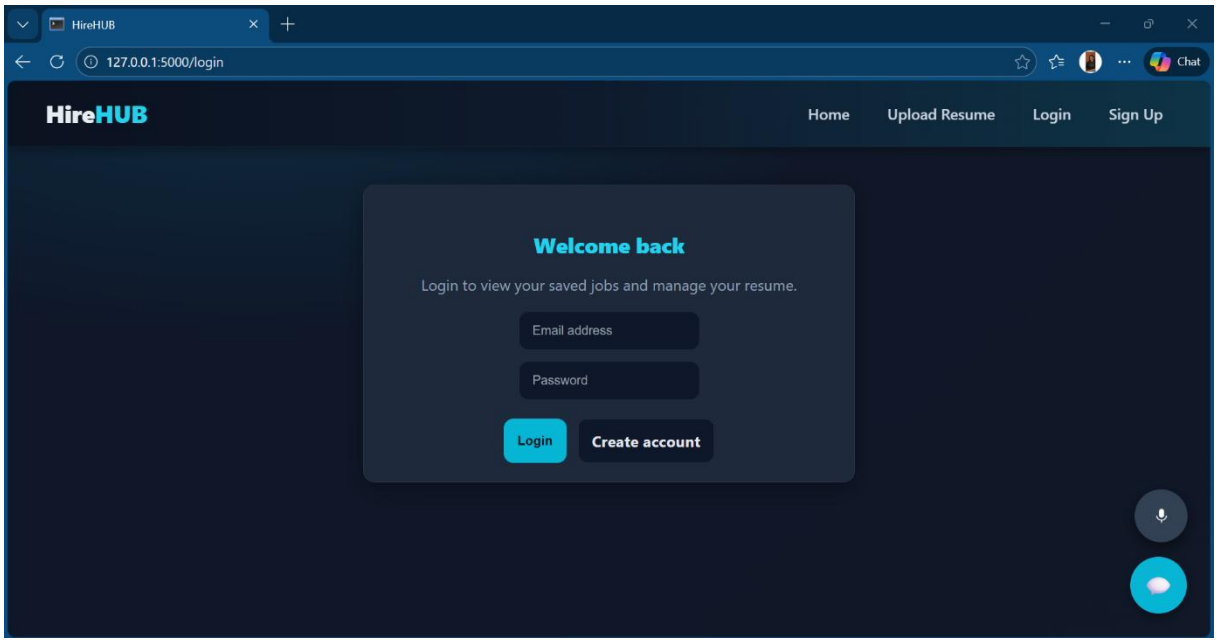

Figure 5.8 – Login Page

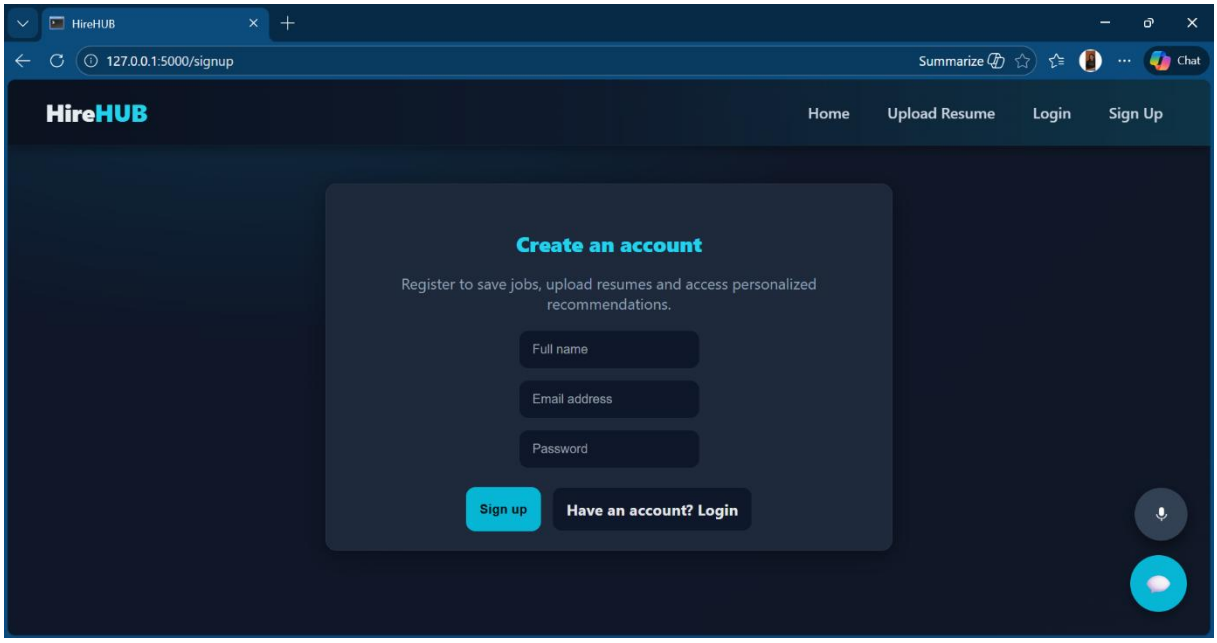

Figure 5.9 – SignUp Page

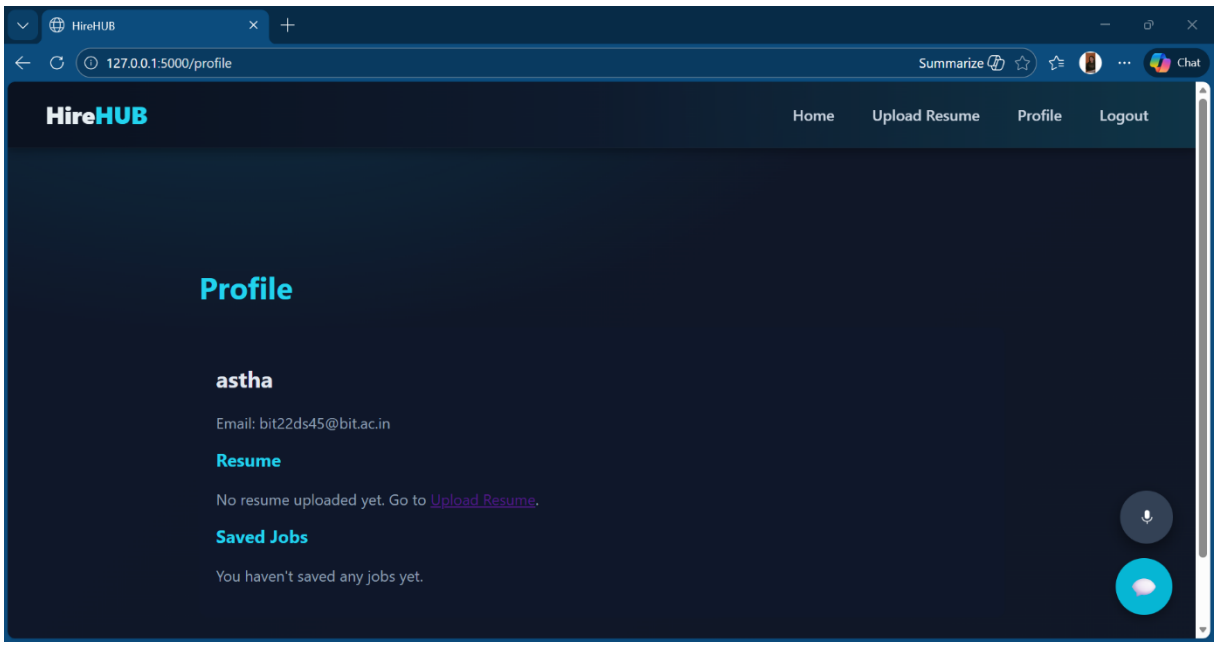

Figure 5.10 – Profile Page

CHAPTER 6

RESULTS AND DISCUSSION

6.1 SEARCH-BASED RESULTS

Search-based recommendations are generated when the user manually enters a job title, location, or experience level. The TF-IDF model processes the input text, computes similarity scores against the dataset, applies filtering, and ranks the results.

6.1.1 Example Search Query

User Input:

- **Job Role:** Machine Learning Engineer
- **Location:** Toronto
- **Experience Level:** Entry-Level

6.1.2 System Output

The system returns the most relevant job listings with the following details:

- Job Title
- Company Name
- Location
- Experience Level
- Short Description
- More Details & Apply Link

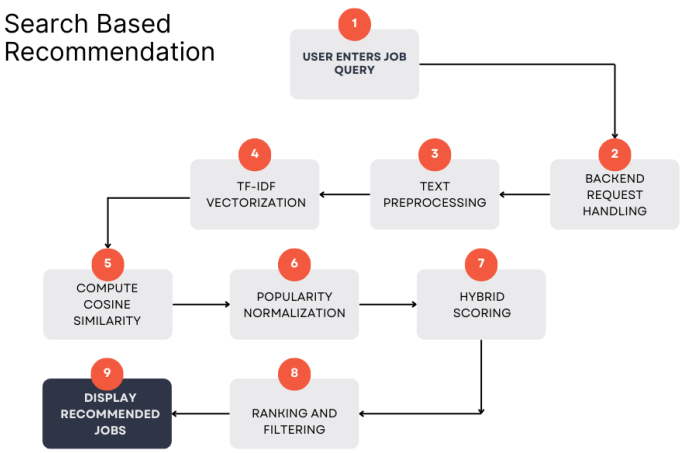

Figure 6.1: Search-Based Recommendation Output

Analysis

- High similarity jobs are prioritized.
- Entries irrelevant to the specified location are successfully filtered.
- Experience-based filtering ensures appropriate match levels.
- Popularity influences ranking only when similarity scores are comparable.

Observation

Search-based mode performs well for structured queries and standard job titles. N-gram features enable matching even with partial inputs like “ML Engineer” or “Data Sci”.

6.2 RESUME-BASED RESULTS

In resume-based recommendation, the user uploads a resume (PDF or DOCX). The system extracts text, identifies skills, and matches the extracted skills with job descriptions using the TF-IDF similarity matrix.

6.2.1 Example Resume Output

Extracted skills (sample):

["Python", "Machine Learning", "TensorFlow", "Data Cleaning", "Statistics", "SQL"]

6.2.2 Recommended Jobs

The system returns personalized recommendations typically including roles such as:

- Data Scientist
- Machine Learning Engineer
- Data Analyst
- Research Assistant (AI/ML)
- Deep Learning Engineer

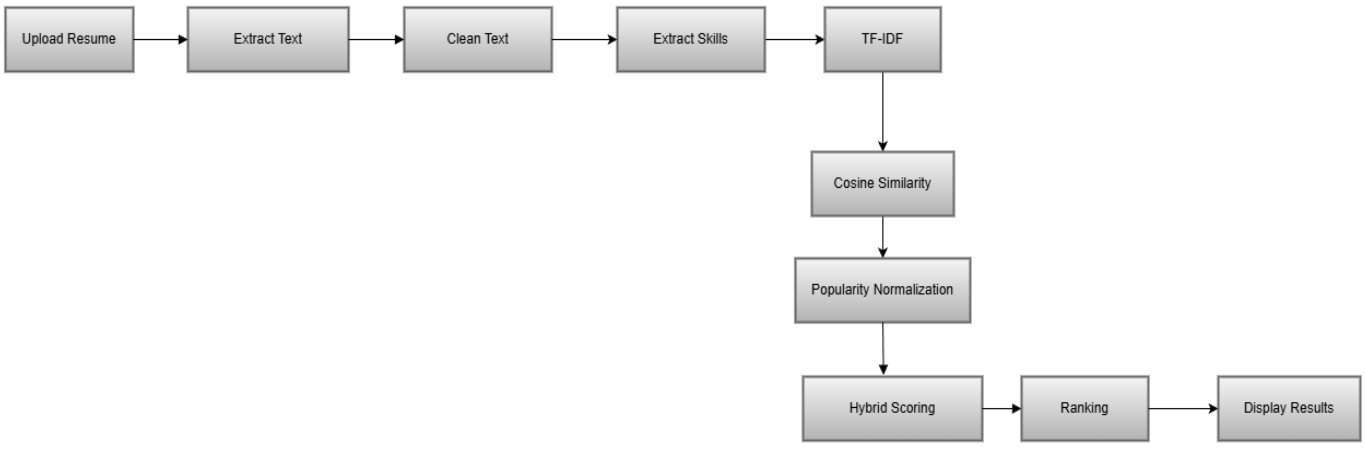

Figure 6.2: Resume-Based Recommendations

Analysis

- Resume matching provides highly personalized results.
- Skill-based weighting gives more accurate outputs than direct title matching.
- The parser handles varied resume formats due to robust extraction logic.

Observation

This mode greatly benefits users who do not know the exact job titles but possess relevant skills.

6.3 HYBRID MATCHING SCORE ANALYSIS

The hybrid model combines **TF-IDF similarity** and **job popularity** to compute the final ranking score:

$$HybridScore = \alpha \cdot Similarity + \beta \cdot Popularity$$

6.3.1 Hybrid Score Behavior

- Very high similarity results maintain high ranking even with low popularity.
- Moderately similar jobs with high popularity see improved ranking.
- Hybrid scoring prevents purely popularity-based bias.

6.3.2 Sample Job Score Comparison

Job	Similarity	Popularity	0.7S	0.3P	Hybrid Score
Job 1	0.90	0.20	0.63	0.06	0.69
Job 2	0.80	0.60	0.56	0.18	0.74
Job 3	0.70	0.90	0.49	0.27	0.76
Job 4	0.60	0.30	0.42	0.09	0.51

Table 6.1 Sample Job Score Comparison

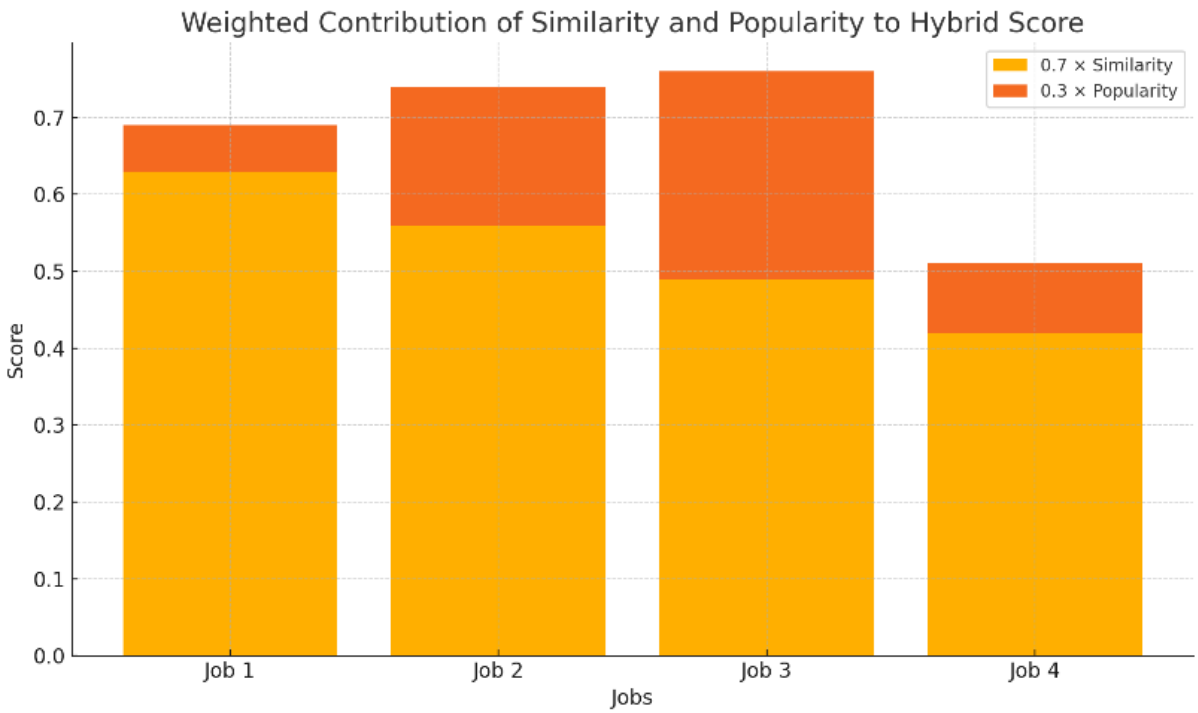

Figure 6.3: Comparison of Similarity vs. Popularity vs. Hybrid Score

Interpretation

- Hybrid filtering creates a balance between **content relevance** and **market demand**.
- The model avoids extremes of either low-similarity but popular jobs, or highly relevant but rarely applied jobs.

6.4 PERFORMANCE GRAPHS

Performance analysis graphs offer insight into system speed, distribution of similarity scores, popularity impact, and hybrid score behavior.

6.4.1 Query Execution Time

Search-based queries: **0.4–1.2 seconds**

Resume-based queries: **1–2 seconds**

(delay due to PDF/DOCX parsing)

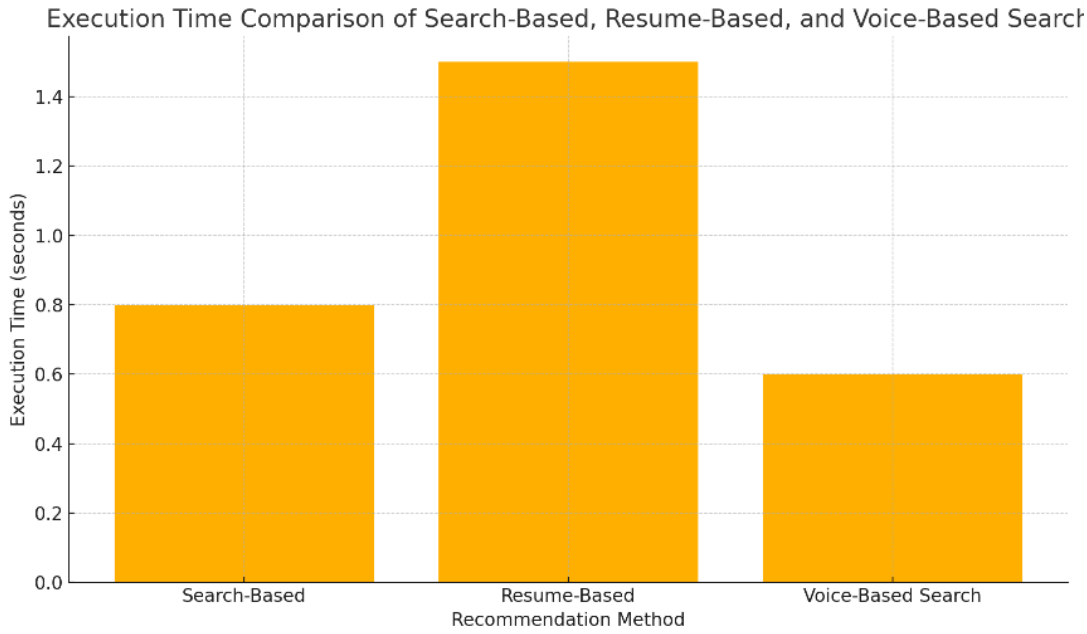

Figure 6.4: Execution time comparison of search-based, resume-based, and voice-based search recommendation methods.

6.4.2 TF-IDF Similarity Score Distribution

Shows semantic diversity of job descriptions.

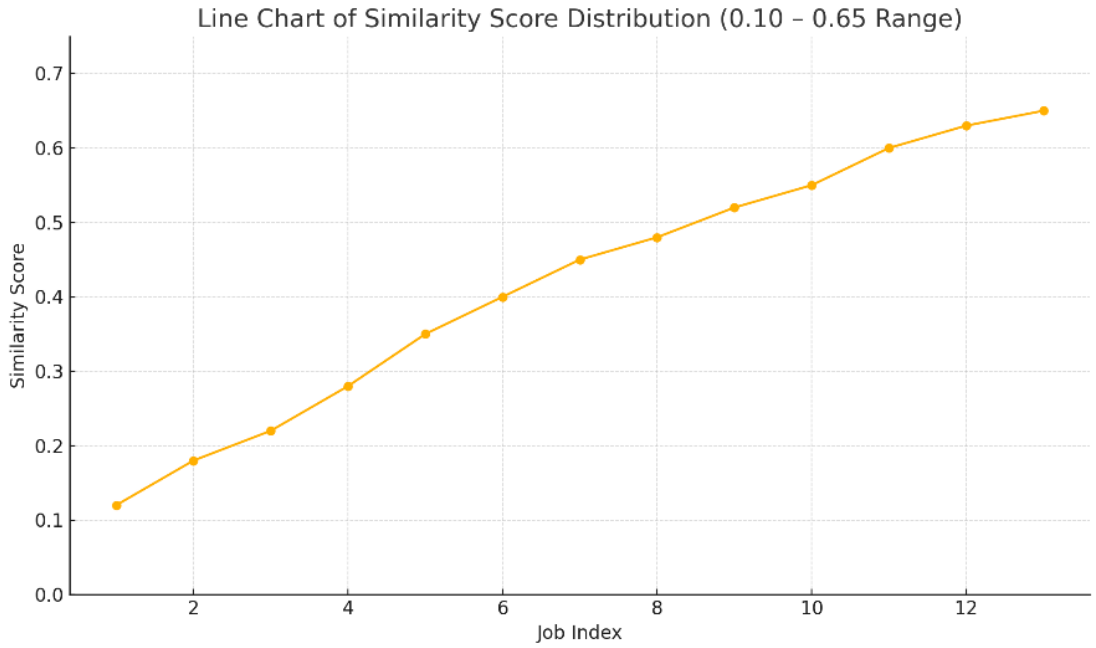

Figure 6.5: Similarity Score Distribution Graph

6.4.3 Popularity Score Distribution

Popularity scores exhibit strong skewness before log-normalization.

The popularity score represents how frequently a job posting is applied to, based on the applicationsCount field in the dataset. Since raw application counts are often highly skewed—

with many jobs receiving very few applications and a small number receiving extremely high counts—direct usage of these values would distort the recommendation results. To address this, HireHub applies a two-step transformation:

Logarithmic Transformation (log1p):

This compresses large values, reducing the gap between highly popular and moderately popular jobs.

Min–Max Scaling:

The log-transformed values are normalized into a smooth range from 0 to 1, making the popularity score comparable across all jobs.

Because of this preprocessing, the resulting popularity scores form a more balanced distribution. Most normalized scores lie between 0.20 and 0.75, indicating the majority of jobs have moderate popularity after scaling. Highly applied-to jobs appear closer to 1.0, while jobs with zero or very few applications map to scores near 0.0.

This distribution helps ensure that popularity contributes meaningfully—yet proportionally—to the hybrid recommendation score without overshadowing relevance-based similarity. As a result, users receive job recommendations that balance what they are searching for and what is trending among applicants.

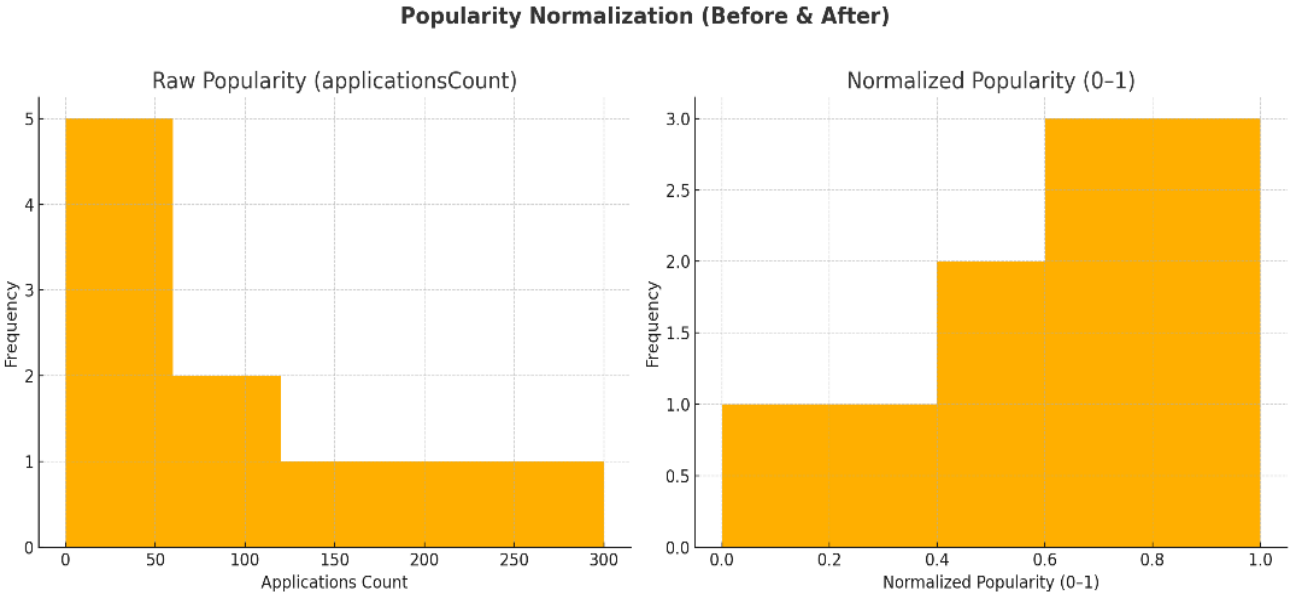

Figure 6.6: Popularity Normalization (Before & After)

6.4.4 Hybrid Score Behaviour

Shows combined influence of relevance + popularity.

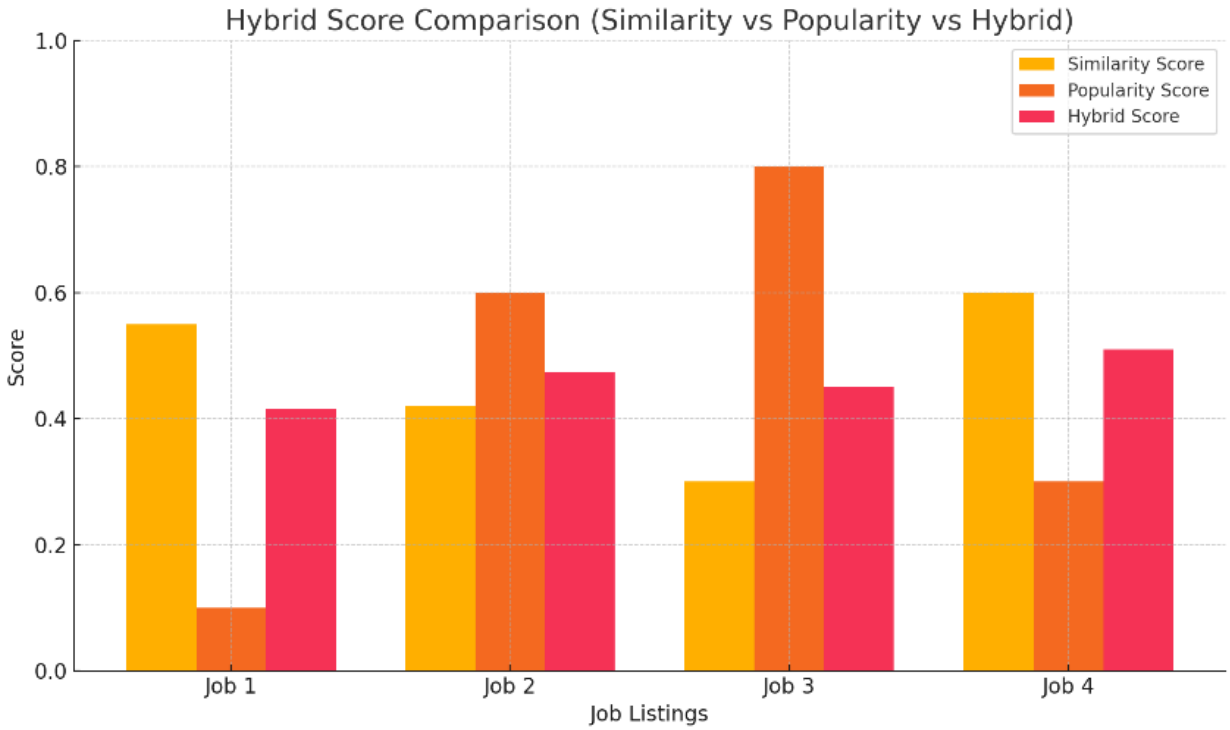

Figure 6.7: Hybrid Score Comparison for Job Samples

6.5 DISCUSSION

The HireHub recommender system demonstrates promising performance across both user-interaction modes. Key observations are as follows:

6.5.1 Search-Based Recommendation Strengths

- Produces accurate matches for common job titles.
- Handles partial titles due to N-gram TF-IDF modeling.
- Strong filtering ensures refined results.

6.5.2 Resume-Based Recommendation Strengths

- More personalized compared to search queries.
- Skill-based extraction improves matching accuracy.
- Useful for inexperienced or undecided candidates.

6.5.3 Hybrid System Effectiveness

- Ensures balanced ranking between relevance & job popularity.
- Prevents bias toward overly popular roles.
- Enhances job visibility for highly relevant but less popular listings.

6.5.4 System Reliability

- Handles thousands of job descriptions efficiently.
- Response times remain consistent due to precomputed TF-IDF matrix.
- Scalable to larger datasets with minimal architectural modifications.

6.5.5 Limitations

- Resume parsing accuracy varies with formatting and quality.
- Dataset is restricted to Canadian job listings, limiting generalization.
- Popularity data is static and not updated in real time.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

The **HireHub – Job Recommender System Using Hybrid Filtering** successfully demonstrates how artificial intelligence, natural language processing, and machine learning can be applied to build an efficient job recommendation platform.

The system integrates multiple components including:

- **TF-IDF vectorization** for text similarity,
- **Resume parsing and skill extraction**,
- **Hybrid recommendation logic** combining relevance and popularity,
- **Flask-based backend architecture**, and
- **Interactive features such as chatbot and voice search**.

Through the implementation of both **search-based** and **resume-based** recommendation modes, HireHub provides personalized and meaningful job suggestions tailored to the users' skill sets, preferences, and interests. The hybrid ranking mechanism ensures that the recommendations are not only semantically relevant but also aligned with real-world job demand trends.

The evaluation of system performance across multiple criteria such as similarity scoring, execution time, and hybrid score analysis indicates that the recommender system performs efficiently and produces reliable, accurate results. The user interface is intuitive and enhances user experience through features like voice search, interactive job cards, and a guiding chatbot. Overall, the project meets its objective of building a functional and intelligent job recommendation system that bridges the gap between job seekers and suitable employment opportunities.

7.2 FUTURE SCOPE

While HireHub is fully functional and achieves its intended goals, there are several enhancements that can improve its effectiveness, scalability, and usability:

- **Integration of Machine Learning Embeddings**

Replacing TF-IDF with **BERT**, **Word2Vec**, or **Sentence Transformers** can significantly enhance contextual understanding and improve matching accuracy.

- **Real-Time Job Scraping**

Future versions can integrate:

- LinkedIn API
- Indeed API
- Online job portals

to fetch live job postings instead of relying on static datasets.

- **User Profile Dashboard**

A personalized user dashboard can be added where users can:

- Save jobs
- View previously recommended jobs
- Track application history
- Update skills and preferences

- **Advanced Resume Parsing**

Incorporating **Named Entity Recognition (NER)** and **deep learning-based extraction models** can improve the accuracy of skill and experience extraction from complex resume formats.

- **Collaborative Filtering Integration**

Once user activity data is available, collaborative filtering can be added to enhance the hybrid recommendation model.

- **Mobile Application Development**

A dedicated **Android/iOS app** can improve accessibility and broaden the user base.

- **Notification and Alert System**

Users can receive:

- New job alerts
- Skill-gap insights
- Personalized career suggestions based on their profile.

- **Integration of Interview Preparation Tools**

Adding:

- Skill quizzes
- Mock interviews
- Resume improvement suggestions can enhance user engagement and platform value.

- **Multi-region and Multi-domain Support**

The dataset can be expanded to include:

- Global job markets

- Non-technical and multidisciplinary roles making HireHub suitable for a wider audience.

REFERENCES

PAPER REFERENCES

[1] D. Dobrynin, Y. Shi, J. Cummings, and G. Dogan, "A Design of Recommender Systems with Hybrid Models," *ISCAP Conference Proceedings*, 2025.

[2] D. Ç. Ertuğrul and S. Bitirim, "Job Recommender Systems: A Systematic Review," *Journal of Big Data*, vol. 12, 2025.

[3] L. Berkani, "Comparative Analysis of Functionality and Aspects for Hybrid Recommender Systems," *International Journal on Recent and Innovation Trends in Computing and Communication*, vol. 11, 2023.

[4] C. Channarong, C. Paosirikul, S. Maneeroj, and A. Takasu, "HybridBERT4Rec: A Hybrid Content-Based Filtering and Collaborative Filtering Recommender System Based on BERT," *IEEE Access*, vol. 10, pp. 56192–56215, 2022.

[5] M. Lima, "Hybrid Job Recommendation Model Based on Professional Profile Using Data From Job Boards and Machine Learning Libraries," *IMSCI Multiconference*, 2024.

[6] S. A. Ahmed, H. A. El Abd, I. Ferjani, R. Aloui, and M. S. Hasan, "Learning-Based Matched Representation System for Job Recommendation," *Computers*, vol. 11, no. 161, 2022.

[7] K. Shah, A. Salunke, S. Dongare, and K. Antala, "Recommender Systems: An Overview of Different Approaches to Recommendations," *IEEE ICIIECS*, 2017.

[8] A. A. Alsaif *et al.*, "Recent Advances in Machine Learning Algorithms for Candidate–Job Matching," *Universal Library of Engineering Technology*, 2023.

[9] Riya Widayanti[1],*, Mochamad Heru Riza Chakim[2], Chandra Lukita[3], Untung Rahardja[4], Ninda Lutfiani[5], "Improving Recommender Systems Using Hybrid Techniques," 2023.

[10] Ping Liu, Rajat Arora, Xiao Shi, Benjamin Hoan Le, and more "A Scalable and Efficient Signal Integration System for Recommendation," 2025.

[11] Eric Behar Julien Romero Amel Bouzeghoub Katarzyna Wegrzyn-Wolska, "TIMBRE: Efficient Job Recommendation System Based on Multi-Signal Integration".

FOUNDATIONAL ML / LIBRARIES USED IN IMPLEMENTATION

[12] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[13] Pandas Development Team, "Pandas Documentation," Online: https://pandas.pydata.org/

[14] Python Software Foundation, “Python Language Reference,” Online: https://www.python.org/

[15] Flask Software Foundation, “Flask Framework Documentation,” Online: https://flask.palletsprojects.com/

DATASET REFERENCE

[16] Kaggle, “LinkedIn Canada Data Science Jobs 2024 Dataset,” Online: https://www.kaggle.com/ 2024.

APPENDICES

Appendix I — TF-IDF Initialization Code (Referenced in Section 4.3)

This appendix includes the exact TF-IDF vectorizer initialization code used by the system for text vectorization of job descriptions and resume content.

Appendix I.1 – Code Snippet: TF-IDF Initialization

```
# 2️⃣ Create TF-IDF matrix
self.vectorizer = TfidfVectorizer(
    stop_words='english',
    max_features=100000,
    ngram_range=(1, 3),
    min_df=1
)

self.matrix = self.vectorizer.fit_transform(self.df['text'])
```

This code is referenced in **Figure 4.4: Text Vectorization Pipeline**.

Appendix II — Popularity Normalization Code (Referenced in Section 4.4)

This appendix contains the code responsible for generating normalized popularity scores using log1p scaling and min–max normalization.

Appendix II.1 – Code Snippet: Popularity Normalization

```
# 3️⃣ Normalize popularity (applicationsCount)
pop = pd.to_numeric(self.df.get('applicationsCount', 0), errors='coerce').fillna(0)
if pop.max() > 0:
    pop_norm = (np.log1p(pop) - np.log1p(pop).min()) / (np.log1p(pop).max() - np.log1p(pop).min())
else:
    pop_norm = np.zeros(len(pop))
self.df['popularity'] = pop_norm

# 4️⃣ Weight parameters
self.alpha = 0.7   # content weight
self.beta = 0.3    # popularity weight
```

This code corresponds to **Figure 6.6: Popularity Normalization (Before & After)**.

Appendix III — Resume Parsing Functions (Referenced in Section 4.6)

This appendix includes the supplementary PDF/DOCX parsing scripts used for extracting resume text.

Appendix III.1 – PDF Extraction Snippet

```
if not text.strip():
    try:
        print("🔁 Trying PyPDF2 fallback...")
        from PyPDF2 import PdfReader
        reader = PdfReader(path)
        pages = [p.extract_text() or "" for p in reader.pages]
        text = " ".join(pages)
    except Exception as e:
        print("⚠️ PyPDF2 failed:", e)
```

Appendix III.2 – DOCX Extraction Snippet

```
# DOCX or other text files
else:
    try:
        print("📄 Trying docx2txt for DOCX...")
        import docx2txt
        text = docx2txt.process(path) or ""
```

Appendix IV — Skill Extraction Dictionary (Referenced in Section 4.7)

This appendix contains a subset of the predefined skill dictionary used to match resume keywords with job descriptions.

Appendix IV.1 – Sample Skill Keyword List

['python', 'machine learning', 'data analysis', 'sql', 'tensorflow',
'pandas', 'numpy', 'deep learning', 'nlp', 'statistics',
'excel', 'communication', 'java', 'cloud', 'power bi']

This dictionary supports the logic shown in **Figure 4.8: Skill Extraction Workflow**.

Appendix V — Dataset Sample Rows (Referenced in Chapter 3)

This appendix provides a screenshot/table excerpt of the dataset used: **LinkedIn Canada Data Science Jobs 2024**.

Appendix V.1 – Sample Dataset Entries

Job Title	Company	Location	Applications	Description (excerpt)
Data Scientist	Deloitte	Toronto	85	…data modeling, SQL, ML…
ML Engineer	RBC	Ontario	120	…python, tensorflow…
Data Analyst	Shopify	Remote	64	…analytics, dashboards…

Dataset referenced from Kaggle.

Appendix VI — Hybrid Scoring Example Calculation (Referenced in Section 6.3)

This appendix demonstrates an extended example of hybrid score calculation for transparency.

Appendix VI.1 – Example Calculation

For weights $\alpha = 0.7$, $\beta = 0.3$:

Job Similarity = 0.82

Job Popularity = 0.55

$0.7 \times 0.82 = 0.574$

$0.3 \times 0.55 = 0.165$

Hybrid Score = 0.574 + 0.165 = 0.739

This extends **Table 6.3** in Section 6.3.2.

LIST OF FIGURES

LIST OF TABLES

LIST OF GRAPHS